Hyperparameter Tuning with Optuna: Bayesian Optimisation

Contents

1. Overview . . . 2
2. Why grid search and random search are not enough . . . 3
   - 2.1 The problem: a random forest for placement . . . 3
   - 2.2 Both searches are blind . . . 3
3. Bayesian search . . . 4
   - 3.1 Accuracy is a function of the hyperparameters . . . 4
   - 3.2 Learning the surface from the trials . . . 5
   - 3.3 Informed search versus blind search . . . 6
4. Optuna's vocabulary . . . 7
5. The Optuna workflow . . . 7
   - 5.1 The objective function . . . 8
   - 5.2 Creating the study and running the trials . . . 9
   - 5.3 The best trial and the final model . . . 10
6. Samplers: Bayesian, random or grid . . . 10
   - 6.1 Bayesian search needs fewer trials . . . 12
7. Visualising a study . . . 13
   - 7.1 Optimisation history . . . 14
   - 7.2 Parallel coordinates and slices . . . 14
   - 7.3 Contour plot . . . 15
   - 7.4 Hyperparameter importances . . . 17
8. Define-by-run: searching over algorithms . . . 17
   - 8.1 The problem: tuning several algorithms . . . 17
   - 8.2 A dynamic search space . . . 18
   - 8.3 Reading the trials as a table . . . 19
9. Other features of Optuna . . . 21
10. Summary . . . 22
11. Sources . . . 22
12. Key terms . . . 23

Where this fits: Pipeline map step 10 (Tune). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	8 Model	9 Evaluate	**10 Tune**	11 Deploy	12 Test	13 Monitor

- **Builds on:** Cross-validation; Hyperparameter tuning.
- **Compare with:** Grid and random search.

1. Overview

Key point: Optuna is a hyperparameter tuning library; instead of trying combinations blindly, it uses the results of past **trials** (G-2016) to choose the next one.

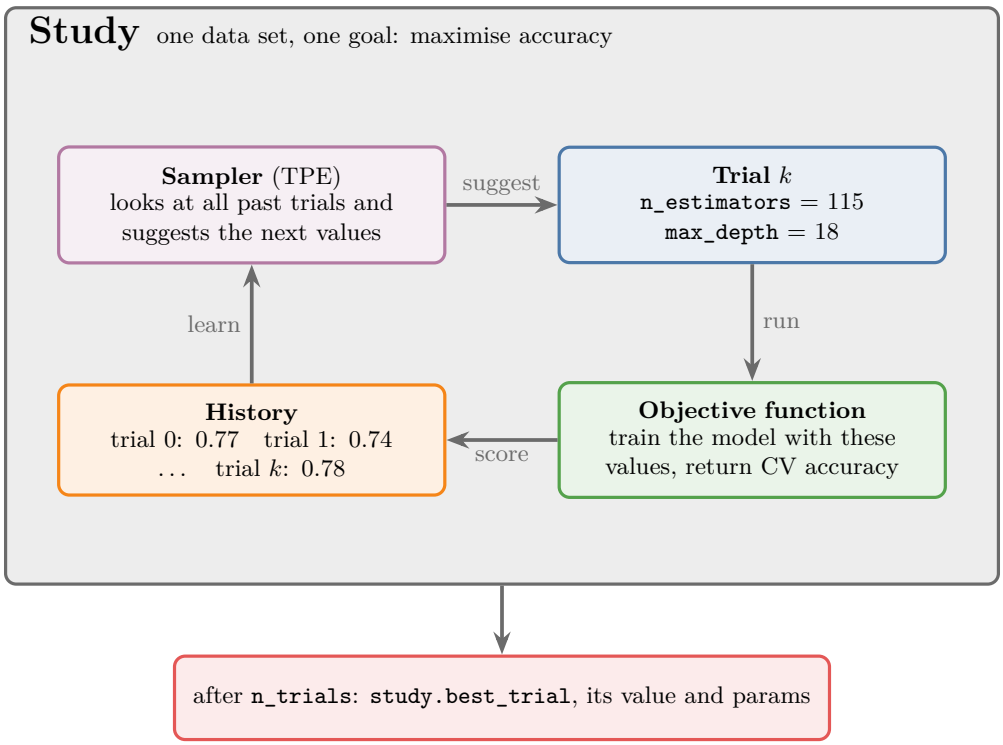


Figure 1: The Optuna loop: a sampler suggests values, a trial runs the objective function, the score joins the history, and the sampler learns from it

Optuna (G-1402) is an open-source Python framework for hyperparameter tuning, used for both machine learning and deep learning. Its main strength is the kind of search it runs: **Bayesian optimisation** (G-270), which learns from every trial where good hyperparameters are likely to be (Figure 1).

This Note covers:

- why grid and random search are not always enough (section 2);

- the idea of Bayesian search (section 3);
- Optuna's vocabulary (section 4) and workflow (section 5);
- its **samplers** (G-1733) (section 6) and plots (section 7);
- its "define-by-run" **search spaces** (G-1756) (section 8).

The Notebook (`ML-128-optuna.ipynb`) runs every Optuna **study** (G-1907) in this Note, and saves them in `data/optuna.db`.

2. Why grid search and random search are not enough

> **Key point:** Grid search trains every combination, which becomes too slow; random search trains a few, which can miss the best one. Neither learns from its earlier **trials** (G-2016).

Hyperparameter (G-910; a setting chosen before training) tuning, as in hyperparameter tuning with a pipeline, tries several hyperparameter values and keeps the best. So far we have done it by **grid search** (G-872) with `GridSearchCV` (see hyperparameter tuning with a pipeline) and `RandomizedSearchCV` (see tuning with GridSearchCV and RandomizedSearchCV); grid search over a random forest and randomized search compare them on a forest.

2.1 The problem: a random forest for placement

> **Key point:** We want the `max_depth` and `n_estimators` that give the highest accuracy, and we cannot know them in advance.

Take the placement data: predict from a student's CGPA and IQ whether they will be placed, with a random forest. We tune two hyperparameters (see the forest-level hyperparameters):

- `max_depth`: how deep each decision tree may grow;
- `n_estimators`: how many trees the forest has.

First we choose a **search space** (G-1756), the values to try, from intuition and domain knowledge: `max_depth` from 1 to 5, and `n_estimators` 50, 100, 150, 200 and 250. These values make a grid of $5 \times 5 = 25$ combinations.

2.2 Both searches are blind

> **Key point:** Grid search pays for every combination; random search can miss the best one; neither learns from its earlier **trials** (G-2016).

Grid search (G-872) would train all 25 models, and the count multiplies with every added value or hyperparameter; random search trains a few, such as 5, and may never draw the best one (the trade-off of tuning with GridSearchCV and RandomizedSearchCV, measured on a forest in randomized search). Both are "blind": the score of one trial never influences which combination is tried next. Figure 2 shows the two on the 5×5 grid: grid search trains every point, random search five points chosen without looking at any score.

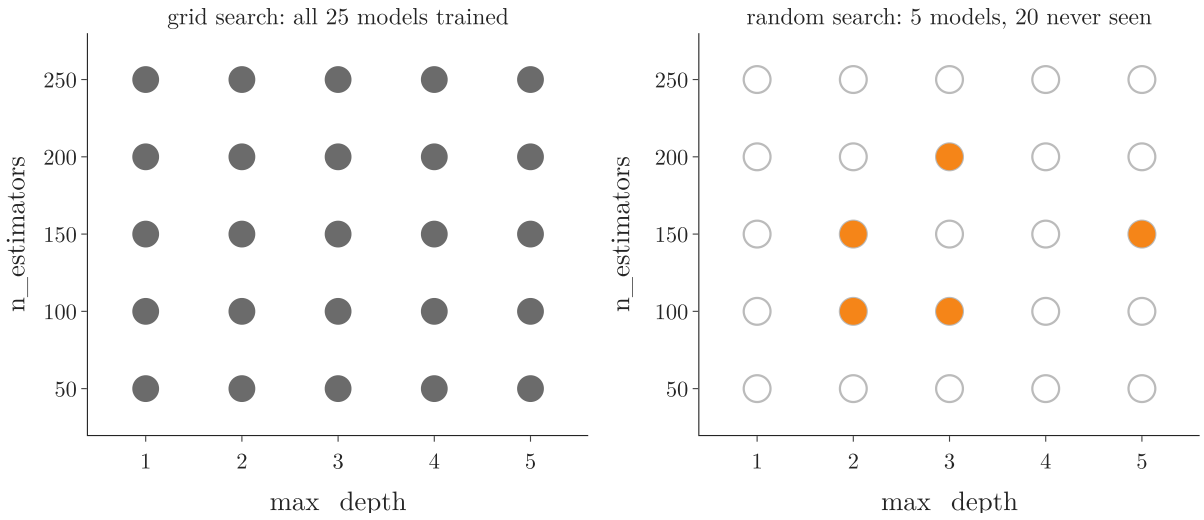


Figure 2: The 5×5 placement grid: grid search trains all 25 combinations (left), random search 5 of them (right)

3. Bayesian search

> **Key point:** Treat accuracy as an unknown function of the hyperparameters, learn its shape from the **trials** (G-2016) so far, and try next where it is most likely to be high.

3.1 Accuracy is a function of the hyperparameters

> **Key point:** Change `max_depth` or `n_estimators` and the accuracy changes, so accuracy = f(max_depth, n_estimators) for some unknown f.

If we change `max_depth` from 5 to 25, the accuracy changes; if we change `n_estimators` from 10 to 100, it changes too. So there is a relationship between the hyperparameters and the accuracy:

$$\text{accuracy} = f(d,\ m)$$

Here d is `max_depth` and m is `n_estimators`.

We do not know f. With two hyperparameters it is a surface over a flat grid: `max_depth` on one axis, `n_estimators` on another, accuracy as the height. If we knew the surface, we would read off its highest point, and with it the best hyperparameters.

3.2 Learning the surface from the trials

> **Key point:** Every **trial** (G-2016) gives one point on the unknown surface; from these points the search guesses the surface and picks the most promising place to try next.

Each trial reveals one point of f. For example, `max_depth` = 5 and `n_estimators` = 100 with accuracy 75% is one point on the surface; `max_depth` = 10 and `n_estimators` = 150 with 69% is another.

To see this, drop `n_estimators` and keep one hyperparameter, so the surface becomes a curve (Figure 3):

1. Try `max_depth` = 5, 10 and 15: accuracies 85%, 65% and 84%. These are three points on the hidden curve.
2. From the three points, guess the whole curve, together with how unsure the guess is in each place: little near the points, a lot far from them.
3. Try next where the guess is high, or where it is very unsure and could be high.
4. Add the new point, update the guess, and repeat.

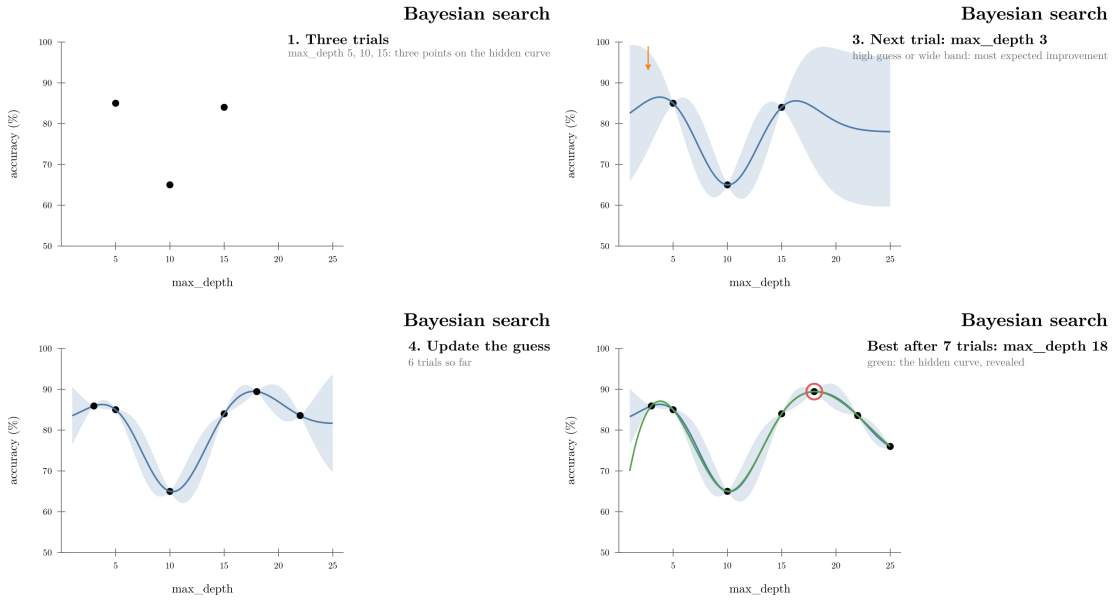


Figure 3: Bayesian search on one hyperparameter: three trials, a guess of the curve with its uncertainty (band), the next trial where the improvement could be largest, and the hidden curve revealed at the end

After a few rounds, the guess matches the hidden curve well near its top, and the best trial is close to the true maximum. Most trials end up in the promising region instead of being spread evenly.

3.3 Informed search versus blind search

> **Key point:** Grid and random search ignore earlier results; Bayesian search uses every earlier result to choose the next **trial** (G-2016).

	Grid search	Random search	Bayesian search
Next trial chosen	in a fixed order	at random	from the results so far
Uses earlier scores	no	no	yes
Trials needed	all combinations	as many as we allow	fewer than random search for the same score

Bayesian optimisation is the name of this informed approach. The method is called Bayesian because it puts a prior belief on the unknown function and updates it with each new result into a posterior, as Bayes' theorem updates a probability with new evidence (see the formula of Bayes' theorem; Shahriari et al. 2016, §II).

> **Extra:** The two parts of **Bayesian optimisation** (G-270) have names. The guess of the curve is the **surrogate model** (G-1927): in Figure 3 a **Gaussian process** (G-832), a model that predicts a value and its uncertainty (a mean μ and a standard deviation σ) at every point. The rule that picks the next trial is the **acquisition function** (G-163); a common one is the **expected improvement** (G-724; Shahriari et al. 2016; Jones et al. 1998).

> **Extra:** Expected improvement, step by step.
>
> 1. **In words:** how much better than the best score so far we expect a point to be, averaging over the surrogate's uncertainty; a point counts as promising if its predicted mean is high, or if its uncertainty is wide enough that it might be high.
> 2. **Formula:** with best score so far f^*, predicted mean μ and standard deviation σ at a point, Φ the normal distribution's cumulative probability and ϕ its density:
>
> $$z = \frac{\mu - f^*}{\sigma}$$
>
> $$\mathrm{EI} = (\mu - f^*)\,\Phi(z) + \sigma\,\phi(z)$$
>
> 3. **Example:** $f^* = 0.785$, $\mu = 0.790$, $\sigma = 0.010$:
>
> $$z = \frac{0.790 - 0.785}{0.010} = 0.5$$
>
> $$\Phi(0.5) = 0.6915$$
>
> $$\phi(0.5) = 0.3521$$

$$\mu - f^* = 0.790 - 0.785 = 0.005$$

$$0.005 \times 0.6915 = 0.00346$$

$$0.010 \times 0.3521 = 0.00352$$

$$\text{EI} = 0.00346 + 0.00352 = 0.0070$$

The two halves are about equal: half of this point's appeal is its high mean, half its uncertainty.

4. Optuna's vocabulary

Key point: A study is the whole tuning task; a trial is one run with one set of values; the objective function scores a trial; the sampler chooses the next values.

Five terms appear in every piece of **Optuna** (G-1402) code (Figure 1):

- **Study** (G-1907): one optimisation session: one dataset, one algorithm (or several), the hyperparameters to tune and the goal. A study is a collection of trials aimed at optimising the objective function.
- **Trial** (G-2016): one run of the objective function with one set of hyperparameter values, for example `max_depth` = 1 and `n_estimators` = 50.
- **Trial parameters:** the hyperparameter values used in a trial, for example `max_depth` = 2 and `n_estimators` = 100.
- **Objective function** (G-1372): the function we want to optimise. The objective takes a trial, builds and trains a model with that trial's values, and returns a score such as accuracy.
- **Sampler** (G-1733): the algorithm that suggests which hyperparameter values to try in the next trial. Optuna's default is **TPE** (G-1995), the Tree-structured Parzen Estimator, a form of **Bayesian optimisation** (G-270) (Bergstra et al. 2011).

Extra: How TPE chooses (Bergstra et al. 2011, §4). TPE sorts the past trials into a "good" group (in Optuna, the best 10%, at most 25 trials) and a "bad" group. For each hyperparameter it estimates how its values are spread in each group, $\ell(x)$ for good and $g(x)$ for bad, and tries next the values where $\ell(x)/g(x)$ is highest: values common among good trials and rare among bad ones. TPE was built for "tree-structured" **search spaces** (G-1756), in which some hyperparameters exist only for some values of another, as in section 8.

5. The Optuna workflow

Key point: Write an **objective function** (G-1372), create a study, call optimize; the best trial holds the best score and values.

We tune a random forest on the Pima diabetes data (see the Pima diabetes example). Each **observation** (one row of the table) is a patient: 768 in all. Each has 8 **features** (input variables, one column each), medical measurements such as glucose and BMI, and the **target**

(the output we predict) is whether the patient has diabetes. We split the data 70/30 into 537 training and 231 test observations. The Notebook (`ML-128-optuna.ipynb`) runs every step of this section.

5.1 The objective function

> **Key point:** Inside the objective, `trial.suggest_int` defines the **search space** (G-1756) and returns this trial's values; the function trains a model and returns its score.

The **objective function** (G-1372) does two things:

1. **Defines the search space** and takes this **trial** (G-2016)'s values from it: `n_estimators` between 50 and 200, `max_depth` between 3 and 20.
2. **Runs the training:** builds a random forest with those values, scores it with 3-fold cross-validation (G-510; the training data is cut into 3 parts, and the model is trained on two and tested on the third, three times; see cross-validation with a pipeline), and returns the mean accuracy.

Python: The objective function.

```
import optuna
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

def objective(trial):
    # 1. the search space: this trial's values
    n_estimators = trial.suggest_int("n_estimators", 50, 200)
    max_depth = trial.suggest_int("max_depth", 3, 20)
    # 2. the training
    model = RandomForestClassifier(
        n_estimators=n_estimators, max_depth=max_depth,
        random_state=42, n_jobs=-1)
    scores = cross_val_score(model, X_train, y_train,
                             cv=3, scoring="accuracy")
    return scores.mean()
```

`trial.suggest_int(name, low, high)` returns a whole number between `low` and `high`, both included, chosen by the **sampler** (G-1733). Every value in the range is allowed, so `n_estimators` can be 51 or 137, not only multiples of 50.

The `trial` object is where the intelligence is hidden: on each call, `suggest_int` asks the sampler, which looks at all earlier trials before answering.

> **Extra:** The other `suggest_` methods: `trial.suggest_float("C", 1e-3, 1e3, log=True)` for decimal values (`log=True` samples on a log scale, where each power of 10 gets the same room, so 0.001 to 0.01 gets as much room as 100 to 1000; **Optuna** (G-1402) docs), and `trial.suggest_categorical("kernel", ["linear", "rbf"])` for a choice from a list.

5.2 Creating the study and running the trials

Key point: `create_study(direction="maximize")` for a score like accuracy, `"minimize"` for a loss; `optimize(objective, n_trials=50)` runs the **trials** (G-2016).

Python: Two lines run the whole search.

```
study = optuna.create_study(
    direction="maximize",
    sampler=optuna.samplers.TPESampler(seed=42))
study.optimize(objective, n_trials=50)
```

`direction="maximize"` because accuracy (or R^2) should go up; for a loss such as log loss or mean squared error we would write `"minimize"`. TPE is the default **sampler** (G-1733), so `sampler=` can be left out; we pass it to fix its `seed`, so every run tries the same values. `optimize` calls `objective` 50 times, each time with a new `trial`.

While it runs, **Optuna** (G-1402) prints one line per trial, such as `Trial 5 finished with value: 0.7747 and parameters: {'n_estimators': 53, 'max_depth': 20}. Best is trial 5 with value: 0.7747.` The first 10 trials are random: **TPE** (G-1995) needs some history before it can model anything, so `TPESampler` samples at random until `n_startup_trials` (default 10) trials have finished (Optuna docs). After that, the trials gather in the regions that scored well. Figure 4 shows this in our **study** (G-1907). In the 10 random start trials, `max_depth` lands on 8 or 9 once; in the 40 TPE trials, 12 times, around the best trial so far (115 trees of depth 8, found at trial 9).

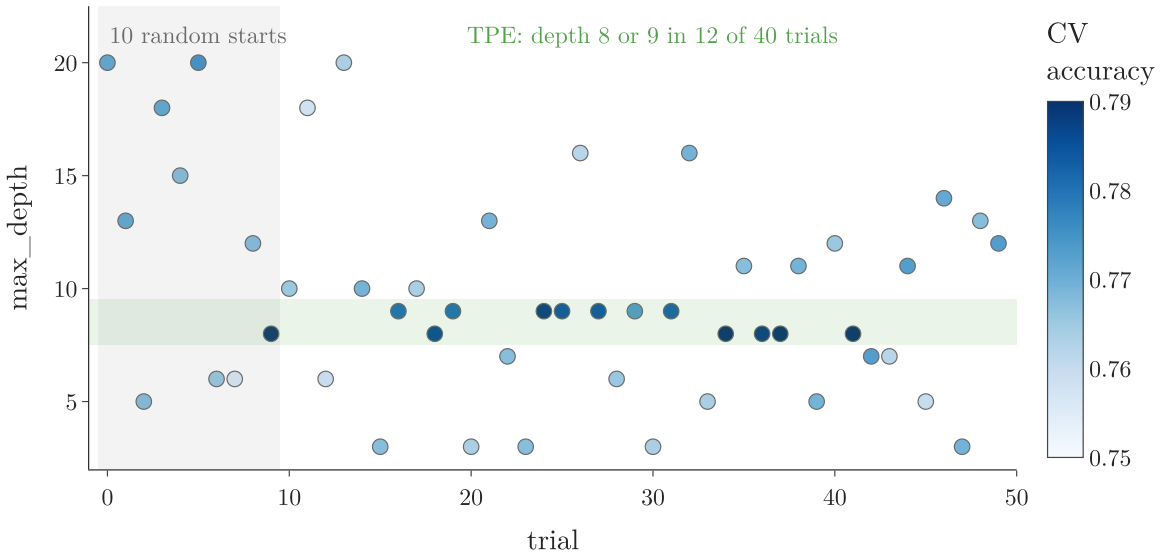


Figure 4: The TPE study's max_depth, trial by trial: random for 10 trials, then returning again and again to depth 8 and 9 (green band); darker dots scored higher

5.3 The best trial and the final model

> **Key point:** `study.best_trial.value` is the best score and `study.best_trial.params` its values; we retrain a model with them and test it once.

Python: Reading the result and testing the best model.

```
best = study.best_trial
best.value     # 0.790, the best mean CV accuracy
best.params    # {'n_estimators': 115, 'max_depth': 8}

model = RandomForestClassifier(**best.params,
                               random_state=42)
model.fit(X_train, y_train)
model.score(X_test, y_test)   # 0.745
```

`**best.params` unpacks the dictionary into keyword arguments: `n_estimators=115, max_depth=8`.

After 50 **trials** (G-2016), the best forest has 115 trees of depth 8, with a cross-validated accuracy of **0.790**. Retrained on all 537 training observations, it scores **0.745** on the 231 test observations. The cross-validated score is the one the search maximised, so it is a little optimistic: the search also picks up the random ups and downs of the folds (Cawley and Talbot 2010). Our data agrees: the same forest on 10 other shuffles of the folds averages 0.772, not 0.790 (Notebook). The test score, 0.745, is the honest one.

6. Samplers: Bayesian, random or grid

> **Key point:** Swapping the **sampler** (G-1733) turns the same study into a TPE search, a random search or a grid search.

The **objective function** (G-1372) stays exactly the same; only the `sampler=` argument changes.

Python: Random search and grid search inside **Optuna** (G-1402).

```
# random search: 50 random combinations
rand = optuna.create_study(
    direction="maximize",
    sampler=optuna.samplers.RandomSampler(seed=42))
rand.optimize(objective, n_trials=50)

# grid search: every combination of an explicit grid
space = {"n_estimators": [50, 100, 150, 200],
         "max_depth": [5, 10, 15, 20]}
grid = optuna.create_study(
    direction="maximize",
    sampler=optuna.samplers.GridSampler(space, seed=42))
```

`grid.optimize(objective, n_trials=16)`
For `GridSampler`, the grid is written outside the objective, and the **trial** (G-2016) values come from it: $4 \times 4 = 16$ trials. With grid or random search inside Optuna, we no longer need scikit-learn's `GridSearchCV` or `RandomizedSearchCV`.

Figure 5 places the trials of the three studies on the same plane, `n_estimators` across and `max_depth` up, one trial per frame. Watch where the dots land:

- **Grid search** fills its 16 fixed points in order, whatever the scores.
- **Random search** scatters its 50 trials evenly over the plane.
- **TPE** starts with the same 10 random trials (both use seed 42). After those, its dots gather around `max_depth` 8 and 9, where the darkest (best) dots are.

The green box is the best region of this **study** (G-1907) (`max_depth` 6 to 10, `n_estimators` up to 120). The counter in each title shows how many trials fall in it: 2 of 16 for the grid, 7 of 50 for random search, 10 of 50 for **TPE** (G-1995).



Figure 5: The trials of the grid, random and TPE studies on the n_estimators × max_depth plane, one trial per frame; darker dots scored higher, the red ring marks the newest trial, and the green box is the best region

Which sampler should we use? The books give a clear answer: Bayesian search reaches a good score in fewer trials than random search, and the saving is largest when each trial is expensive and the good settings fill only a small part of the **search space** (G-1756) (Shahriari et al. 2016, §I; Bergstra et al. 2011). Section 6.1 tests this under exactly those conditions.

6.1 Bayesian search needs fewer trials

> **Key point:** Averaged over 20 runs, **TPE** (G-1995) reaches in 20 trials the score that random search needs about 42 trials to reach.

Think of a treasure hunt. If gold lies everywhere, any random dig finds some, and clues do not help much. If the gold lies in one small patch, a hunter who uses the clues from earlier digs finds it far sooner than one who digs at random. Bayesian search is the hunter who uses the clues. Our random forest is the first kind of field: almost any depth and tree count scores about the same (see the Extra below). So we tune an RBF-kernel SVM (a support vector machine with a curved boundary, see the kernel trick in SVM) on the same diabetes data, over its two settings `C` and `gamma`, each from very small to very large on a log scale; only a narrow band of this space scores well. We run each **sampler** (G-1733) 20 times with different seeds, 50 trials each, and average the best score so far after every trial (Figure 6).

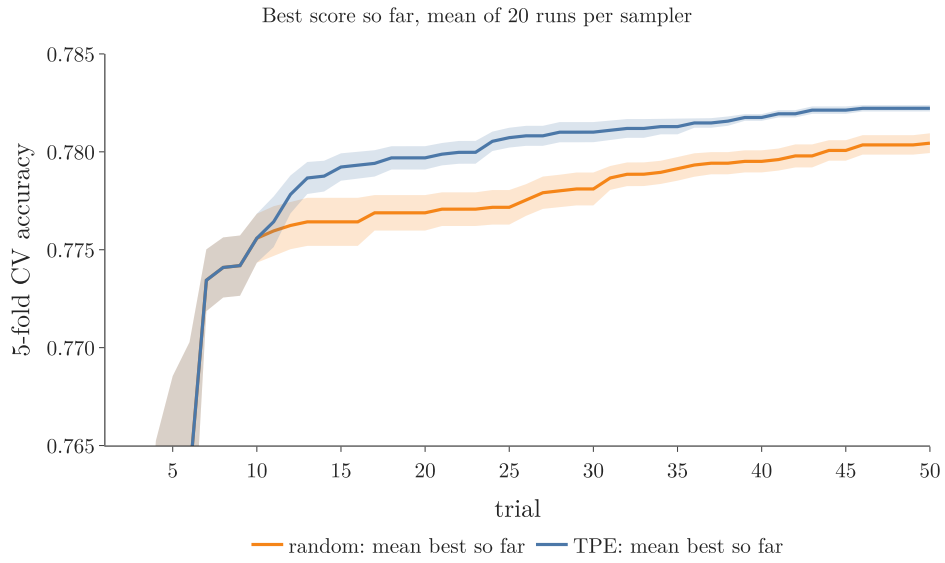


Figure 6: Best 5-fold CV accuracy so far, averaged over 20 runs of each sampler on the SVM search; the bands are ± one standard error

How to read Figure 6: across is the number of **trials** (G-2016) run, up is the best score found so far (not the latest score), averaged over 20 separate runs; the shaded band shows the spread between runs. A curve that is higher at the same trial count has found a better setting sooner. The first 10 trials are identical, because TPE starts at random (section 5.2). From then on, the TPE curve rises faster. After 20 trials TPE averages 0.780 and random search 0.777; random search reaches 0.780 only at trial 42. After 50 trials TPE is level with or ahead of random search in 19 of the 20 runs (Notebook). Fewer trials for the same score is exactly what Bayesian optimisation promises.

Extra: The three samplers on the forest objective of section 5, one **study** (G-1907) each:

Sampler	Trials	Best CV	Mean CV	Best values	Test
Grid	16	0.784	0.768	50 trees, depth 15	0.749
Random	50	0.793	0.769	56 trees, depth 8	0.740
TPE	50	0.790	**0.771**	115 trees, depth 8	0.745

TPE placed its trials best: 10 of its 50 landed in the best region (`max_depth` 6 to 10, `n_estimators` up to 120), against 7 for random search and 2 for the grid, so its average trial scored highest. The best scores, though, cannot rank the samplers here. On 537 observations, cross-validated accuracy is noisy: one fixed forest, scored on 10 shuffles of the folds, moves between 0.760 and 0.795 (Notebook). The three best CV scores (0.784, 0.790, 0.793) differ by less than that, and the three test accuracies all lie between 0.74 and 0.75. A flat score surface plus noisy scores is the field where clues do not help, which is why section 6.1 uses the SVM, fixed folds and 20 repeated runs.

Extra: Other samplers in `optuna.samplers` (**Optuna** (G-1402) docs) include `GPSampler` (Gaussian-process **Bayesian optimisation** (G-270), as in Figure 3), `CmaEsSampler` (an evolution strategy for many numeric hyperparameters) and `NSGAIISampler` (for studies with several objectives at once).

7. Visualising a study

Key point: `optuna.visualization` draws interactive Plotly charts of a **study** (G-1907): the history, where the trials went, and which hyperparameters mattered.

Python: The five main plots; each returns a Plotly figure.

```
from optuna.visualization import (
    plot_optimization_history, plot_parallel_coordinate,
    plot_slice, plot_contour, plot_param_importances)

plot_optimization_history([grid, rand, study]).show()
plot_parallel_coordinate(study).show()
plot_slice(study).show()
plot_contour(study).show()
plot_param_importances(study).show()
```

A Plotly figure can be saved as an image with `fig.write_image("history.png")`. The figures below are these plots, with fonts and colours changed to match this Note.

7.1 Optimisation history

> **Key point: Trial** (G-2016) number against score, with the best so far as a line; when the line stops rising, more trials are not paying off.

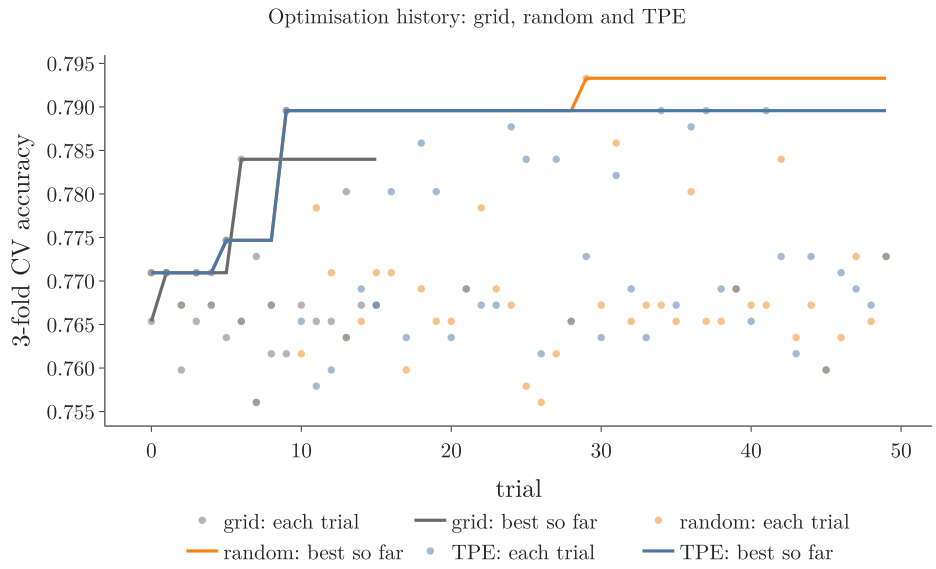


Figure 7: Optimisation history of the three studies: each trial (dots) and the best so far (lines)

Reading the chart: each dot is one trial, with the trial number across and its score up; the line for each **study** (G-1907) is the highest score found so far, so it can only stay flat or step up. Figure 7 puts the three studies on one chart. TPE's best, 0.790, came at trial 9, still among its first 10 random trials; later it matched that score three more times (trials 34, 37 and 41), all in the same region. Random search found its 0.793 at trial 29. No study improved its best score after trial 29, so on this problem the last 20 trials added nothing; a flat history like this one helps us choose `n_trials` next time.

7.2 Parallel coordinates and slices

> **Key point:** One vertical axis per quantity and one line per trial, coloured by score; where the dark lines bunch, the **sampler** (G-1733) found a promising range.

Each **trial** (G-2016) is a short row of numbers: its `n_estimators`, its `max_depth` and its score. The best trial of our TPE **study** (G-1907) is the row (115, 8, 0.790). A **parallel coordinates** plot gives each of the three quantities its own vertical axis, side by side, and draws one trial as one line that crosses every axis at that trial's value. The row (115, 8, 0.790) becomes a line through 115 on the first axis, 8 on the second and 0.790 on the third. With 50 trials there are 50 lines; darker lines are higher scores, so a bundle of dark lines shows which values the good trials share.

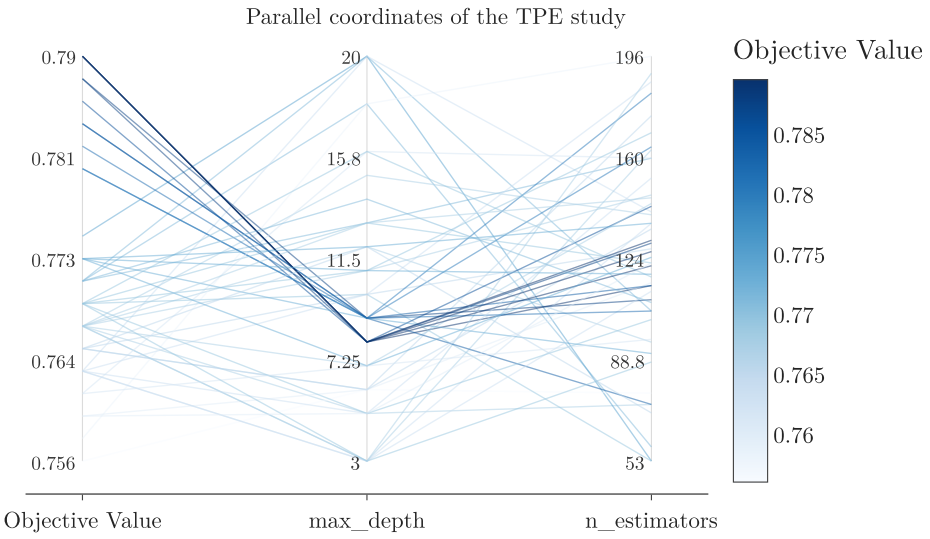


Figure 8: Parallel coordinates of the TPE study: each line is a trial, darker is a higher score

In Figure 8, the darkest lines all run through `max_depth` 8 to 9 and on to `n_estimators` between about 110 and 145. A **slice plot** (`plot_slice`) shows the same thing one hyperparameter at a time: each hyperparameter against the score, one dot per trial, so dense regions show where the sampler spent its trials.

7.3 Contour plot

> **Key point:** Two hyperparameters on the axes and the score as coloured contours: the best region of the **search space** (G-1756) at a glance.

The accuracy is a function of the two hyperparameters (Section 3.1), so with `n_estimators` and `max_depth` on the floor and the accuracy as the height, the 50 trials of the TPE **study** (G-1907) are 50 points of a surface. Between the trials the surface is filled in by linear interpolation (straight lines between neighbouring trials). Figure 9 draws it with the trials as black dots; the red dot is the best trial, $(115, 8)$ with accuracy 0.790, while the lowest **trials** (G-2016) score about 0.758. Seen from above, it is a contour map: each line joins points of the same estimated accuracy, and darker means higher. Lines close together mean the accuracy changes fast there; the darkest rings are the best region.

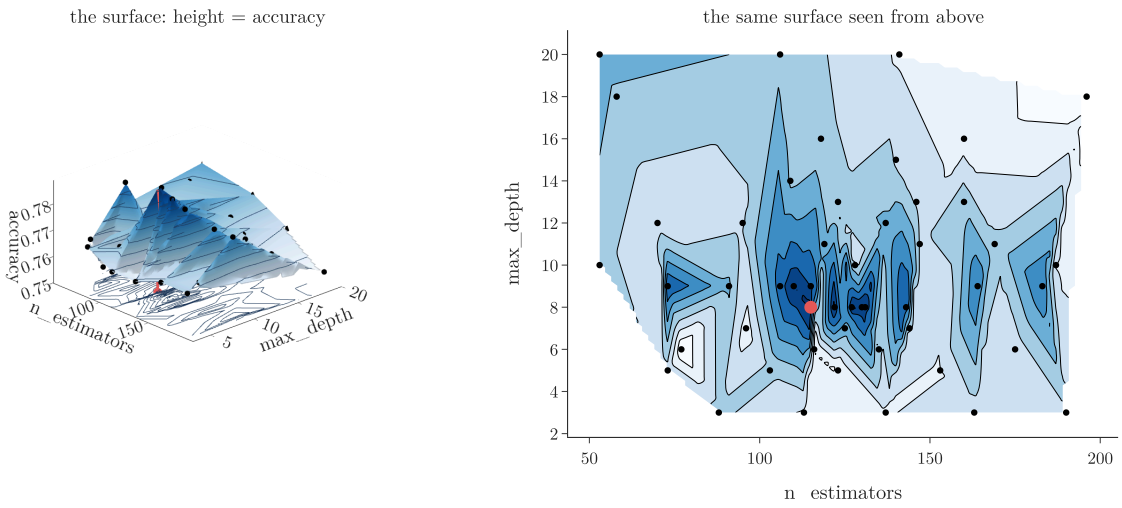


Figure 9: The accuracy surface of the TPE study (left) and the same surface seen from above (right). Black dots: the 50 trials. Red dot: the best trial.

Figure 10 is **Optuna** (G-1402)'s own contour plot of the same surface.

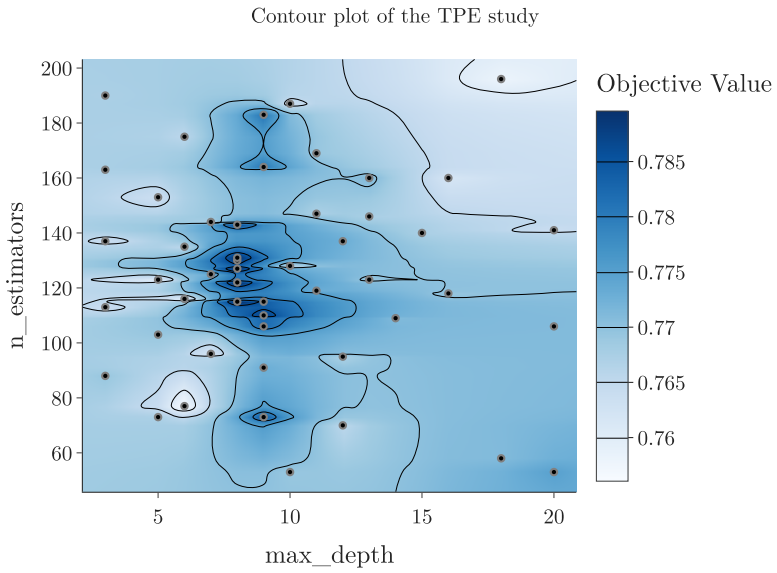


Figure 10: Contour plot of the TPE study: max_depth against n_estimators, darker is a higher score; dots are trials

Figure 10 shows the dark region around `max_depth` 8 to 9 and `n_estimators` 105 to 135, and the dots show that **TPE** (G-1995) placed many of its trials there. Away from that band of depths, the colours are pale whatever the number of trees.

7.4 Hyperparameter importances

> **Key point:** How much each hyperparameter affected the score in this **study** (G-1907); here `max_depth` matters far more than `n_estimators`.

Each bar is one hyperparameter; the longer the bar, the more the score changes when that hyperparameter changes. The bars add up to 1.

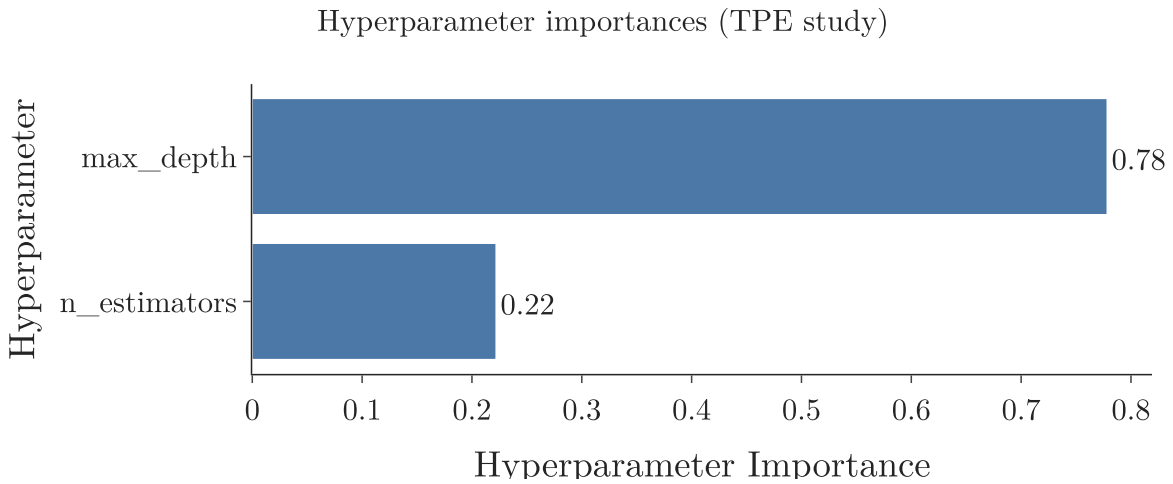


Figure 11: Hyperparameter importances of the TPE study

Figure 11 gives `max_depth` a **hyperparameter importance** (G-908) of 0.78 and `n_estimators` 0.22; the values add up to 1. So on this data, when time is short, `max_depth` is the hyperparameter to tune carefully.

8. Define-by-run: searching over algorithms

> **Key point:** Because the **search space** (G-1756) is built while the objective runs, the algorithm itself can be a hyperparameter, and each algorithm can have its own hyperparameters.

8.1 The problem: tuning several algorithms

> **Key point:** Tuning four algorithms one after another means four separate searches.

On a new dataset we rarely know which algorithm will do best: logistic regression, an SVM, a random forest, gradient boosting? Normally we tune each one separately and compare their best scores, which means four full searches.

8.2 A dynamic search space

> **Key point:** Make `classifier` a categorical hyperparameter, then use `if`/`elif` to suggest only that classifier's hyperparameters.

In **Optuna** (G-1402), the search space is not fixed in advance: it is created by the `suggest_` calls as the **objective function** (G-1372) runs. Building the space this way is called **define-by-run** (G-575). So the first suggestion can pick the algorithm, and the later suggestions can depend on that choice. A **search space** (G-1756) whose hyperparameters depend on other hyperparameters is a **dynamic search space** (G-653), also called a conditional search space.

Python: One **study** (G-1907) over three algorithms.

```python
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

def make_model(trial):
    name = trial.suggest_categorical(
        "classifier",
        ["SVC", "RandomForest", "GradientBoosting"])
    if name == "SVC":
        c = trial.suggest_float("C", 1e-3, 1e3, log=True)
        kernel = trial.suggest_categorical(
            "kernel", ["linear", "rbf"])
        return make_pipeline(StandardScaler(),
                             SVC(C=c, kernel=kernel))
    if name == "RandomForest":
        return RandomForestClassifier(
            n_estimators=trial.suggest_int(
                "n_estimators", 50, 200),
            max_depth=trial.suggest_int("max_depth", 3, 20),
            random_state=42)
    return GradientBoostingClassifier(
        n_estimators=trial.suggest_int(
            "gb_n_estimators", 50, 200),
        learning_rate=trial.suggest_float(
            "learning_rate", 0.01, 0.3, log=True),
        max_depth=trial.suggest_int("gb_max_depth", 2, 6),
        random_state=42)

def objective(trial):
    return cross_val_score(make_model(trial),
                           X_train, y_train, cv=3).mean()
```

```
study = optuna.create_study(
    direction="maximize",
    sampler=optuna.samplers.TPESampler(seed=42))
study.optimize(objective, n_trials=100)
study.best_trial.params
# {'classifier': 'RandomForest',
#  'n_estimators': 134, 'max_depth': 8}
```

Each `if` branch suggests only its own algorithm's hyperparameters, so an SVC **trial** (G-2016) never draws a `max_depth`. The SVM gets a `StandardScaler` in a pipeline because SVMs are not scale invariant (their result changes when a feature is measured in other units; scikit-learn user guide, SVM tips). Building the model in its own function, `make_model`, lets us rebuild the winner later (section 8.3).

The TPE **sampler** (G-1733) chooses the classifier like any other hyperparameter. After 100 trials, the best is a random forest with 134 trees of depth 8: cross-validated accuracy **0.788**, test accuracy **0.749**. One study replaced three.

8.3 Reading the trials as a table

Key point: `study.trials_dataframe()` gives one row per **trial** (G-2016); grouping by the classifier column shows how often each algorithm was tried and how well it did.

Python: The trials as a pandas DataFrame, and the winner rebuilt.

```
df = study.trials_dataframe()
# columns: number, value, datetime_start, datetime_complete,
# duration, params_C, params_classifier, ..., state
df.groupby("params_classifier")["value"].agg(
    ["count", "mean", "max"])

# FixedTrial answers every suggest_ call from a dictionary
best = make_model(
    optuna.trial.FixedTrial(study.best_trial.params))
best.fit(X_train, y_train).score(X_test, y_test)  # 0.749
```

A hyperparameter that a trial never suggested (such as `params_C` in a random forest trial) is NaN ("not a number", an empty cell) in that row.

Classifier	Trials	Mean CV accuracy	Best CV accuracy
Random forest	51	0.775	**0.788**
Gradient boosting	36	0.765	0.780
SVC	13	0.735	0.780

The counts are uneven because **TPE** (G-1995) learns. Counting trials in blocks of 20 (the last block covers the final 40 trials) shows how:

Trials	Random forest	Gradient boosting	SVC
0 to 19	8	8	4
20 to 39	1	17	2
40 to 59	2	11	7
60 to 99	**40**	0	0

Figure 12 replays the **study** (G-1907) trial by trial. Watch the colours: all three appear early, orange (gradient boosting) takes over in the middle, and blue (random forest) fills the last 40 trials.

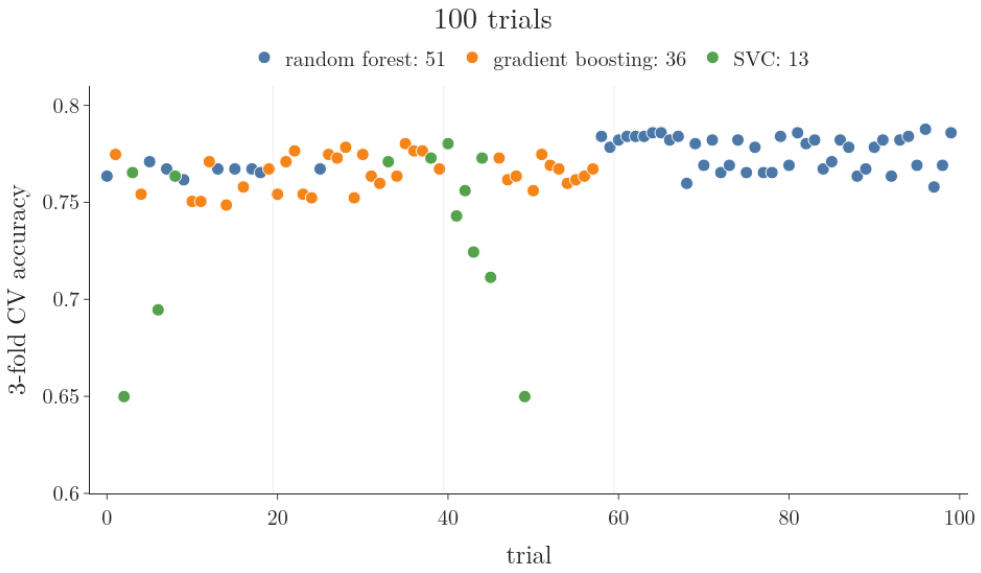


Figure 12: The define-by-run study, 5 trials per frame: each dot is a trial, coloured by its classifier

At first it explores all three. For a while it favours gradient boosting, then it settles on the random forest and spends all of the last 40 trials tuning it. A random **sampler** (G-1733) would have split the trials about evenly, and tuned the winner far less.

> **Extra:** The SVC's low mean and high best go together: its score depends strongly on `C`. Its 13 trials run from 0.650 (`C` of 0.012 or less) to 0.780 (`C` = 0.62). 0.650 is the share of patients without diabetes in the training data, the score of a model that always predicts "no diabetes" (Notebook).
> So the ranking of algorithms depends on how well each one is tuned: at a poor `C` the SVC scores only 0.650, no better than always predicting "no diabetes"; at `C` = 0.62 it reaches 0.780, level with the best gradient boosting trial. This is why comparing algorithms at their default settings can mislead; each should get the same tuning effort first. Melis et al. (2017) found the same in language models: once every model was tuned with the same large search, a standard LSTM (an older kind of neural network for text) beat

newer models that had looked better.

9. Other features of Optuna

Key point: Optuna (G-1402) can spread trials over many machines, works with the main ML and DL libraries, and can stop bad trials early.

- **Distributed optimisation:** several processes or machines can run trials of the same **study** (G-1907) at once, sharing the results through a database, so the search finishes sooner.
- **Integrations:** Optuna works with scikit-learn, Keras, PyTorch, XGBoost, LightGBM and with MLflow (which records experiments).
- **Machine learning and deep learning alike:** the **objective function** (G-1372) can train anything that returns a score.

Python: A study stored in a database, so several processes can work on it.

```
study = optuna.create_study(
    study_name="rf-diabetes", direction="maximize",
    storage="sqlite:///study.db", load_if_exists=True)
study.optimize(objective, n_trials=50)
```

Running the same lines in several terminals at once makes each one add **trials** (G-2016) to the same study.

Extra: Pruning (G-1586) stops unpromising trials early. In a neural network trained for many epochs, the objective reports the score after each epoch with `trial.report(score, epoch)`; if `trial.should_prune()` returns `True` (with the default `MedianPruner`: the trial is worse than the median of earlier trials at the same epoch; Optuna docs), the objective raises `optuna.TrialPruned()` and the study moves on.

10. Summary

	Grid search	Random search	Bayesian search (Optuna TPE)
Chooses the next trial	in order	at random	from all earlier results
Cost	all combinations	`n_trials`	`n_trials`
Risk	too slow	misses the best	can follow noise on small data
Trials to reach 0.780 (SVM, average of 20 runs)	not run	42	**20**
Average best after 20 trials (SVM, 20 runs)	not run	0.777	**0.780**
In Optuna	`GridSampler`	`RandomSampler`	`TPESampler` (default)

- Bayesian optimisation treats the score as an unknown function of the hyperparameters, guesses it from the **trials** (G-2016) so far, and tries next where the improvement could be largest, so most trials land in the promising region instead of being spread evenly.
- Optuna's vocabulary: study, trial, trial parameters, **objective function** (G-1372), sampler; every piece of Optuna code is built from these five.
- The workflow: an objective with `trial.suggest_*`, then `create_study(direction=...)`, `optimize(objective, n_trials=...)`, `best_trial.params`, retrain and test, because the best cross-validated score is a little optimistic and only a test on unseen data shows the real score.
- `optuna.visualization` gives Plotly charts: optimisation history, parallel coordinates, slice, contour, importances, so we see whether more trials still pay off, where the good region lies and which hyperparameters matter.
- Bayesian search needs fewer trials when the good region is small: averaged over 20 runs on an SVM search, TPE reached in 20 trials what random search needed 42 trials for. On the forest, whose scores are nearly flat, the three **samplers** (G-1733) ended within the noise of cross-validation.
- Define-by-run lets one **study** (G-1907) choose the algorithm and its hyperparameters together: TPE spent its last 40 of 100 trials on the random forest, so one study replaced separate searches and the winning algorithm got the most tuning.
- So **Optuna** (G-1402) tunes by learning from past trials, which saves trials when each one is expensive and the good settings are rare.

11. Sources

Built from

- CampusX, "Hyperparameter Tuning using Optuna | Bayesian Optimization using Optuna", YouTube, https://www.youtube.com/watch?v=E2b3SKMw934

Other references

- Shahriari et al. 2016: B. Shahriari, K. Swersky, Z. Wang, R. P. Adams and N. de Freitas, *Taking the Human Out of the Loop: A Review of Bayesian Optimization*, Proceedings of the IEEE 104(1), 2016.
- Jones et al. 1998: D. R. Jones, M. Schonlau and W. J. Welch, *Efficient Global Optimization of Expensive Black-Box Functions*, Journal of Global Optimization 13, 1998.
- Bergstra et al. 2011: J. Bergstra, R. Bardenet, Y. Bengio and B. Kégl, *Algorithms for Hyper-Parameter Optimization*, NeurIPS 2011.
- Cawley and Talbot 2010: G. C. Cawley and N. L. C. Talbot, *On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation*, Journal of Machine Learning Research 11, 2010.
- Melis et al. 2017: G. Melis, C. Dyer and P. Blunsom, *On the State of the Art of Evaluation in Neural Language Models*, arXiv:1707.05589, 2017, abstract.
- Optuna docs: Optuna 5.0 API reference: `TPESampler` (`n_startup_trials`, `default_gamma`), `Trial.suggest_float` (`log`), `optuna.samplers`, `MedianPruner`.
- scikit-learn user guide, SVM tips: scikit-learn user guide, "Support Vector Machines", Tips on Practical Use.

12. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Trial (hyperparameter tuning) (G-2016)	One run of a tuner (Optuna, Keras Tuner) with one set of hyperparameter values: one model built, trained and scored.
Optuna (G-1402)	A Python framework for hyperparameter tuning that uses the results of earlier tries to pick the next values to test (Bayesian optimisation).
Bayesian optimisation (G-270)	Tuning that builds a model of how the score depends on the settings (hyperparameters) and uses all earlier trials to choose the next one.
Sampler (G-1733)	In Optuna, the algorithm that suggests the next trial's hyperparameter values.
Search space (G-1756)	The ranges or lists of values each hyperparameter may take during tuning; the tuner only tries values from it, and each added value multiplies the number of combinations.

Term	Meaning
Study (G-1907)	In Optuna, one full search for the best settings: a collection of trials, each scoring one set of values, aimed at optimising the objective function.
Surrogate model (G-1927)	A cheap stand-in for the unknown score function, built by Bayesian optimisation from the trials so far; it is used to guess which hyperparameter values to try next.
Gaussian process (G-832)	A model that predicts a value and its uncertainty at every point; a common choice of surrogate, the stand-in model that Bayesian optimisation searches.
Acquisition function (G-163)	The rule that picks the next trial using the stand-in model of the score (the surrogate), such as expected improvement.
Expected improvement (G-724)	A score that picks which point Bayesian optimisation tries next: how much better than the best score so far a candidate point is expected to be, judged from the surrogate model's predicted mean and uncertainty. The point with the highest expected improvement is tried next.
TPE (G-1995)	Optuna's default way to pick the hyperparameter values to try next from the results of past trials (its sampler, a form of Bayesian optimisation); TPE stands for Tree-structured Parzen Estimator.
Hyperparameter importance (G-908)	How much each hyperparameter affected the score in a study; the values add up to 1.
Define-by-run (G-575)	Building the search space while the objective function runs, so it can depend on earlier choices.
Dynamic search space (G-653)	A search space in which some hyperparameters exist only for some values of another, such as the algorithm; the tuner then suggests only settings that make sense for the choice already made.
Pruning (Optuna) (G-1586)	Stopping an unpromising trial early, before its training finishes.
Objective function (Optuna) (G-1370)	The function a search optimises: it takes a trial's hyperparameter values and returns a score.

Term	Meaning
Hyperparameter (G-910)	A setting of an algorithm chosen before training, such as a tree's `max_depth`.
Grid search (G-872)	Training a model for every combination of listed settings and keeping the best by cross-validation.
Objective function (G-1372)	The quantity an algorithm tries to make as large or as small as possible.
Cross-validation (G-510)	Testing a model by training and testing it several times on different parts of the training data.
Pruning (G-1587)	Stopping a tree early or cutting it back so it does not overfit.